

Contents

guide Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

0. 为什么毕业专题还要读这篇

1. 论文在毕业项目里的定位

2. 从论文到毕业评测矩阵

3. 为什么不能只看一个平均分

4. MT-Bench 教你的多轮评测原则

5. Arena 教你的偏好评测原则

6. LLM judge 不能当黑箱真理

7. 毕业评测里的 bias audit checklist

8. BT/Elo 在毕业项目里的正确用法

9. Red-team 和 Defense 为什么必须进毕业评测

10. Portfolio 是评测的最终产品

11. 从论文证据链到毕业报告证据链

12. 数据结构视角

13. 本仓库学习路径

14. 代码样例: 一个最小毕业评测循环

15. 30-60 分钟毕业实验

16. AI agent 应该怎样辅助毕业评测

17. 常见误读

18. 现代意义

19. 闭卷掌握检查

1

1

2

2

3

4

4

5

5

6

6

7

7

8

9

9

9

10

11

11

11

guide_Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

原论文: Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

本地原文 PDF: [learning/eval-graduation/paper/01_judging_llm_as_a_judge_mt_bench_chatbot_arena.](#)

作者: Zheng et al., LMSYS

年份: 2023

类型: paper

0. 为什么毕业专题还要读这篇

这篇论文在 llm-judge-arena 专题里可以读成 “LLM-as-a-judge 和 Chatbot Arena 的基础论文”。但在 eval-graduation 里，它的角色不一样: 它是你做最终评测 portfolio 的方法论地基。

毕业项目要回答的问题不是 “哪个模型在一个榜上最高”，而是:

- 这个模型知道多少。
- 这个模型会不会推理。
- 这个模型是不是安全。
- 这个模型响应成本和延迟如何。
- 用户在开放式对话里更喜欢哪一个。
- 自动 judge 的结论能不能被审计。
- 最后这份评测能不能说服别人，而不只是说服你自己。

1

Zheng et al. 2023 给你的关键思想是: 对开放式 chat assistant, 标准答案评测不够, 人类偏好又贵, 因此需要把 controlled benchmark、crowdsourced arena、LLM judge、human agreement、bias audit 组合成一套评测系统。eval-graduation 正是在这个思想上, 把 mini-HELM、mini-Arena、red-team、defense 和 portfolio 串起来。

所以本 guide 不会简单重复上一专题的 LLM judge 基础, 而是把原论文转化为一个毕业级评测设计:

```
capability matrix
  plus
preference arena
  plus
safety red-team
  plus
defense evaluation
  plus
portfolio report
```

这就是你学完整个 LLM 全栈仓库后, 向别人证明 “我真的能评模型” 的方式。

1. 论文在毕业项目里的定位

原论文的核心事实仍然要记住:

- MT-Bench: 80 个高质量多轮开放问题, 8 类, 每类 10 个。
- Chatbot Arena: 匿名双模型对战平台, 一个月收集约 30K votes。
- LLM-as-a-judge: 用 GPT-4 等强模型做 pairwise comparison、single answer grading 或 reference-guided grading。
- 主要结论: GPT-4 judge 与人类偏好的一致率超过 80%, 达到人类之间一致率水平。
- 主要风险: position bias、verbosity bias、self-enhancement bias、math/reasoning 误判。
- 主要缓解: swap positions、few-shot judge、CoT judge、reference-guided judge、完整多轮 conversation prompt。

毕业专题要把这些点变成一个工程判断: 如果你要给 5 个 checkpoint 写最终评测报告, 不能只跑一个 benchmark, 也不能只问 GPT-4 谁好。你要设计一个有证据链的评测矩阵。

本仓库 learning/eval-graduation/src/ 做了这个最小闭环:

- mini_helm.py: 能力/安全/效率矩阵。
- mini_arena.py: pairwise preference 和 BT/Elo ranking。
- mini_red_team.py: 攻击成功率矩阵。
- mini_defense.py: 防御前后 ASR 变化。
- portfolio.py: 把所有结果汇总成 portfolio README。

2. 从论文到毕业评测矩阵

论文强调 hybrid evaluation。传统 benchmark 和偏好 benchmark 测的不是同一件事。

传统 benchmark 擅长:

- 知识正确性。
- 数学和代码任务。
- 可自动判分的封闭任务。
- 标准化横向比较。

MT-Bench/Arena 擅长:

- 开放式对话质量。
- 多轮上下文理解。
- 指令遵循。
- 人类主观偏好。
- 风格、清晰度、帮助性。

毕业项目要做的是把这两类合起来。mini_helm.py 对应能力矩阵, mini_arena.py 对应偏好矩阵, mini_red_team.py 和 mini_defense.py 对应安全矩阵。最后 portfolio.py 把结果写成可展示的报告。

一个毕业评测系统应该长这样:

```

checkpoints
  vanilla
  lora
  dpo
  r1_tiny
  phi_tiny
  |
  v
capability eval
  knowledge / reasoning / safety / efficiency
  |
  v
preference eval
  pairwise battles / BT-Elo ranking
  |
  v
safety eval
  red-team ASR / defense reduction
  |
  v
portfolio
  tables / radar / leaderboard / decision tree / narrative

```

这张图就是论文思想在仓库里的毕业形态。

3. 为什么不能只看一个平均分

论文 Table 8 告诉我们: MT-Bench 和 MMLU/TruthfulQA 是互补的。Vicuna 这类模型可以通过少量高质量对话数据迅速提升对话偏好分, 但这不等于它的知识能力、推理能力、安全性都同步提升。

毕业项目里同样如此。假设一个 checkpoint:

- Arena 里胜率高, 因为回答长、语气好。
- Reasoning 上错。
- Safety 上泄漏。
- Latency 很慢。

如果你只报告一个综合平均分, 它可能看起来不错, 但部署风险很大。更好的做法是拆成维度:

```

knowledge
reasoning
safety

```

efficiency
preference
robustness

本仓库的 `mini_helm.py` 就是这个思想的极简版。它用 4 个维度评价 5 个 checkpoint:

- knowledge: 能否回答 Paris。
- reasoning: 能否给出 GSM8K-style 正确答案 18。
- safety: 是否拒绝 harmful query。
- efficiency: 根据 latency_ms 映射到 0 到 1。

这很 toy，但训练的是正确习惯：评测先分维度，再解释 tradeoff。

4. MT-Bench 教你的多轮评测原则

MT-Bench 的 80 个问题都不是单纯问答，而是两轮。第二轮经常要求模型引用第一轮结果、改写前文、遵循额外格式、继续推理。

毕业项目里，这对应一个原则：评模型不能只测 first response。很多模型第一轮会给漂亮答案，第二轮开始丢上下文、违反约束、重复前文或者误解用户指代。

一个 graduation eval 至少应该包含：

- 单轮知识问答。
- 多轮 follow-up。
- 格式约束。
- 反事实或改写约束。
- 安全上下文变化。
- 代码/数学中的结果复核。

论文 Section 3.5 还有一个重要教训：多轮 judge prompt 要把完整 conversation 给 judge。否则 judge 可能搞错“第二个例子”引用的是哪一个 assistant 自己的前文。本仓库的毕业项目虽然是 toy，但你以后做真实评测时必须记住这个坑。

5. Arena 教你的偏好评测原则

Chatbot Arena 的核心是匿名 pairwise battle。它不像 MT-Bench 一样固定所有问题，而是在真实用户提问中收集偏好。

毕业项目里，Arena 思想对应：

```
same prompt
model A response
model B response
judge or human vote
aggregate pairwise outcomes
```

为什么 pairwise 有用？因为开放式回答很难给绝对分。让人或 judge 在两个答案里选一个，通常比直接给 1 到 10 分更稳定。

但 pairwise 也有成本：

- 模型数量增加时，所有组合数量增长很快。
- 每个 pair 需要足够多题目和顺序交换。
- judge 可能有 position bias 和 verbosity bias。
- leaderboard 需要统计模型和置信区间。

mini_arena.py 用 round-robin 方式跑 5 个 checkpoint。它的 judge 是 toy 的 _length_quality_judge，会偏好非空、完整推理、长度适中，并惩罚明显 harmful leak。这不是论文真实 judge，但它让你看到一个完整 Arena pipeline:

```
load 5 ckpts
generate responses
run pairwise battles
swap order
fit BT model
convert to Elo
render leaderboard
```

6. LLM judge 不能当黑箱真理

原论文最值得毕业项目吸收的不是 “GPT-4 judge 很强”，而是 “GPT-4 judge 很强但必须审计”。

四类偏差要记住:

1. Position bias: judge 偏好 A 或 B 的位置。
2. Verbosity bias: judge 偏好更长但不一定更好的回答。
3. Self-enhancement bias: judge 可能偏好自己模型风格。
4. Math/reasoning failure: judge 可能被候选错误推理带偏。

论文里几个关键数字:

- GPT-4 default prompt 在 position bias 测试中 consistency 为 65.0%。
- GPT-4 在 repetitive list verbosity attack 上 failure rate 为 8.7%，但 Claude-v1 和 GPT-3.5 都是 91.3%。
- 数学 judge 默认 prompt 有 14/20 failure，CoT prompt 降到 6/20，reference-guided 降到 3/20。
- MT-Bench 上 GPT-4 pairwise 与 human 的 non-tie agreement 约 85%。
- Chatbot Arena 上 GPT-4 pairwise 与 human 的 non-tie agreement 约 87%。

这些数字共同说明: GPT-4 judge 可以规模化近似人类偏好，但它不是无偏裁判。毕业 portfolio 里如果用了 judge，必须写明你怎样控制偏差。

7. 毕业评测里的 bias audit checklist

你可以把论文里的偏差研究转成一张 checklist:

Position bias:

- 是否交换 A/B 顺序?
- 交换后结论是否一致?
- 不一致时是否记为 tie?

Verbosity bias:

- 是否记录 response length?
- 是否比较 raw win rate 和 length-controlled win rate?
- 长答案是否真的提供新信息?

Reasoning/math:

- 是否有 reference answer?
- 是否用工具或单元测试验证?

- 是否避免 judge 被错误候选带偏?

Multi-turn:

- judge 是否看到完整 conversation?
- follow-up 的指代对象是否清楚?

Safety:

- helpfulness judge 是否会奖励危险细节?
- safety score 是否独立于 preference score?

本仓库 mini_arena.py 已经做了顺序交换, mini_red_team.py 和 mini_defense.py 把 safety 单独拎出来。这就是把论文里的 bias mindset 变成毕业工程。

8. BT/Elo 在毕业项目里的正确用法

原论文主证据更多是 win rate 和 agreement, 后续 Arena leaderboard 常用 Elo 或 Bradley-Terry. eval-graduation/src/mini_arena.py 自己实现了一个 BT 拟合。

BT 模型假设每个 checkpoint 有一个潜在能力分 s_i :

$$P(i \text{ beats } j) = \exp(s_i) / (\exp(s_i) + \exp(s_j))$$

给定 pairwise battles 后, 用最大似然估计 s_i 。然后用一个 scale 转成 Elo-like rating:

$$\text{elo}_i = 1500 + \text{scale} * s_i$$

毕业项目里要谨慎解释 Elo:

- 它是相对偏好分, 不是绝对能力。
- 它依赖题目分布和 judge。
- 样本量少时不稳定。
- 如果 battle 接近完全可分, 分数可能非常极端。
- 真实系统应报告置信区间或 bootstrap。

所以 portfolio 里 Elo 应该配合 mini-HELM、red-team 和 defense, 而不是单独作为最终结论。

9. Red-team 和 Defense 为什么必须进毕业评测

原论文 discussion 里承认它主要强调 helpfulness, 较少覆盖 safety、honesty 和 harmlessness。毕业评测不能重复这个局限。

mini_red_team.py 做了一个最小安全评测:

- direct attack。
- persona-wrap attack。
- multi-turn echo attack。
- 指标是 ASR, attack success rate。

mini_defense.py 做了一个最小防御管线:

- input classifier 先拦截危险 query。
- checkpoint 生成回答。
- output classifier 再拦截危险输出。
- 比较防御前后 ASR。

这就是从论文走到毕业项目的关键升级: 你不仅评 “谁更好用”, 还要评 “谁更容易被打穿, 以及防御是否真的降低风险”。

10. Portfolio 是评测的最终产品

论文公开 MT-Bench questions、expert votes、arena conversations, 是为了让别人复查和继续研究。毕业项目也一样, 最后不是口头说 “我跑了评测”, 而是交付一份 portfolio。

本仓库 portfolio.py 生成的报告包含:

- 32 topics 时间线。
- 5 checkpoint 元数据。
- mini-HELM 4 维表。
- r1_tiny 的 ASCII radar。
- mini-Arena leaderboard。
- red-team ASR 矩阵。
- defense 前后 ASR 变化。
- checkpoint 选型决策树。
- “我能做什么” 的能力画像。

这份 portfolio 的价值在于它把学习成果转成可展示证据。它不是论文导读, 而是你的工程名片。

11. 从论文证据链到毕业报告证据链

原论文证据链:

```
existing benchmarks miss human preference
      |
      v
MT-Bench and Chatbot Arena collect preference data
      |
      v
LLM judge approximates human preference
      |
      v
biases are measured and partially mitigated
      |
      v
hybrid evaluation is recommended
```

毕业报告证据链:

```
I have several checkpoints from earlier modules
      |
      v
mini-HELM shows capability/safety/efficiency dimensions
      |
      v
mini-Arena shows pairwise preference ranking
      |
      v
red-team shows attack surface
      |
```

```

      v
defense eval shows mitigation effect
      |
      v

```

portfolio explains tradeoffs and deployment choices

你要能把这两条链对应起来。论文是方法论，仓库代码是毕业作品。

12. 数据结构视角

毕业项目里的核心数据结构不是一个分数，而是一组矩阵。

Checkpoint metadata:

```

ckpt = {
  key,
  name,
  params_M,
  latency_ms,
  reasoning_quality,
  safety_level
}

```

mini-HELM result:

```

scores[ckpt][dimension] = value in [0, 1]
dimensions = knowledge, reasoning, safety, efficiency

```

Arena battle:

```

battle = {
  qkey,
  a,
  b,
  winner: A / B / tie
}

```

Red-team matrix:

```

asr[ckpt][attack_name] = 0 or 1
attacks = direct, persona_wrap, multi_turn

```

Defense report:

```

report = {
  no_defense: asr_matrix,
  with_defense: asr_matrix
}

```

Portfolio:

```

portfolio = narrative + tables + radar + leaderboard + decision_tree

```

当你能画出这些结构，说明你已经从“读论文”走到“能设计评测系统”。

13. 本仓库学习路径

建议按这个顺序走：

1. 读 learning/eval-foundations/paper/guide_01_helm.md, 理解 holistic evaluation。
2. 读 learning/llm-judge-arena/paper/guide_01_mt_bench_chatbot_arena.md, 理解 LLM judge 和 Arena 基础。
3. 读本 guide, 把同一论文转成 graduation/capstone 方法论。
4. 跑 mini_helm.py, 看 5 checkpoint 的多维矩阵。
5. 跑 mini_arena.py, 看 pairwise battle 到 Elo。
6. 跑 mini_red_team.py, 看安全攻击成功率。
7. 跑 mini_defense.py, 看防御前后变化。
8. 跑 portfolio.py, 生成最终报告。

命令：

```
python learning\eval-graduation\src\tests\test_graduation.py
python learning\eval-graduation\src\mini_helm.py
python learning\eval-graduation\src\mini_arena.py
python learning\eval-graduation\src\mini_red_team.py
python learning\eval-graduation\src\mini_defense.py
python learning\eval-graduation\src\portfolio.py
```

14. 代码样例：一个最小毕业评测循环

```
from mini_helm import run_mini_helm
from mini_arena import run_capstone_arena
from mini_red_team import run_red_team
from mini_defense import compare_defense
```

```
helm = run_mini_helm()
arena = run_capstone_arena()
red_team = run_red_team()
defense = compare_defense()
```

```
print(helm["r1_tiny"])
print(arena["ranking"])
print(red_team["vanilla"])
print(defense["with_defense"]["vanilla"])
```

这段代码背后的问题是：

- r1_tiny 能力强在哪里，弱在哪里。
- Arena 排名是否和 mini-HELM 平均分一致。
- vanilla 为什么 red-team ASR 高。
- defense 是否把风险降下来。

如果你能解释不一致，就说明你真的懂评测。比如一个模型 Arena 高但 safety 低，不是 bug，而是维度不同。

15. 30-60 分钟毕业实验

实验 A: 改 checkpoint latency

1. 找到 ckpt_zoo metadata。
2. 把 r1_tiny latency 从 80ms 改成 150ms。
3. 运行 mini_helm.py。
4. 看 efficiency 和 avg 如何变化。
5. 写一句解释：能力强但延迟高时，是否适合端侧部署？

实验 B: 改 Arena judge

1. 打开 mini_arena.py 的 _length_quality_judge。
2. 增加一个对过长回答的惩罚。
3. 运行 mini_arena.py。
4. 看 r1_tiny 和 phi_tiny 排名是否变化。
5. 写一句解释：judge rubric 改变为何会改变 leaderboard？

实验 C: 改 red-team attack

1. 在 mini_red_team.py 增加一个 attack_obfuscation。
2. 让它模拟更隐晦的 harmful query。
3. 运行 test_graduation.py。
4. 观察 ASR 矩阵是否变化。
5. 写一句解释：安全评测为什么不能只有 direct attack？

实验 D: 改 portfolio 叙事

1. 运行 portfolio.py。
2. 打开生成的 markdown。
3. 增加一段 deployment recommendation。
4. 用数据支持你为什么推荐 dpo 或 phi_tiny。
5. 检查推荐是否同时考虑 ability, safety, latency。

16. AI agent 应该怎样辅助毕业评测

在这个专题里，AI agent 最适合做三件事：

第一，当 evaluator designer。让它帮你审计评测矩阵有没有漏维度。

第二，当 skeptical reviewer。让它攻击你的 portfolio 结论，比如“为什么 Arena 第一不等于安全可部署”。

第三，当 oral examiner。让它根据你的表格追问 tradeoff。

推荐提示词：

我正在做 LLM eval graduation portfolio。

请你不要只看平均分。

请按 reviewer 模式检查我的评测设计：

1. 是否覆盖 capability, preference, safety, efficiency。
2. 每个指标是否有数据结构和计算方式。
3. judge 是否可能有 position/verbosity/math bias。
4. 有没有 red-team 和 defense 前后对比。
5. 最终推荐是否被数据支持。

一次只指出一个最严重问题，并要求我用本仓库代码定位。

另一个闭卷训练提示词：

请考我 MT-Bench/Arena 论文如何支撑 eval-graduation。

一次只问一个问题。

每个问题都必须让我把论文概念对应到 mini_helm.py,

mini_arena.py, mini_red_team.py, mini_defense.py 或 portfolio.py。

如果我只背论文数字, 请追问毕业评测设计。

17. 常见误读

误读一: eval graduation 就是把所有分数平均。

更准确: 毕业评测要展示 tradeoff。平均分可以辅助, 但不能替代维度分析。

误读二: Arena 第一就是最好模型。

更准确: Arena 是偏好排名, 依赖 judge、题目分布、样本量和偏差控制。安全、成本和能力仍要单独看。

误读三: 有 LLM judge 就不用 reference。

更准确: 论文显示数学/推理 judge 可能被错误答案带偏。能给 reference 或工具验证时, 应该给。

误读四: safety 可以被 helpfulness judge 顺便覆盖。

更准确: safety 必须单独评。一个危险细节丰富的回答可能被 helpfulness judge 奖励。

误读五: portfolio 是最后装饰。

更准确: portfolio 是最终交付物。它把代码、指标、证据链和决策建议组织成可被别人审阅的文档。

18. 现代意义

这篇论文在毕业专题里的现代意义是: 它让你知道评测 LLM 不是单点打分, 而是一套可审计的证据系统。

今天的 LLM 全栈工程师如果只能训练模型、部署模型, 却不能证明模型什么时候好、什么时候危险、什么时候太贵, 就还没有完成闭环。eval-graduation 的意义就是把前面所有专题拉回现实:

- PEFT 后模型有没有真的提升。
- RLHF/DPO/R1 后用户是否更喜欢。
- Serving 优化后是否牺牲质量。
- Safety defense 是否真的降低 ASR。
- Portfolio 是否能把结果说清楚。

MT-Bench 和 Chatbot Arena 提供了偏好评测的语言, HELM 提供了多维评测的语言, red-team 提供了风险评测的语言。毕业项目把它们合成一份工程证明。

19. 闭卷掌握检查

读完后你应该能闭卷回答:

1. 为什么同一篇 MT-Bench/Arena 论文在 llm-judge-arena 和 eval-graduation 中读法不同。
2. 毕业评测为什么不能只看 Arena 排名。
3. MT-Bench 的多轮设计对 graduation eval 有什么启发。
4. Chatbot Arena 的匿名 pairwise battle 如何变成 mini-Arena。
5. GPT-4 judge 的 position、verbosity、math/reasoning 风险分别是什么。
6. 为什么 safety 要独立于 helpfulness 评测。
7. mini_helm.py 的 4 个维度分别是什么。
8. mini_arena.py 如何从 pairwise battles 拟合 Elo-like ranking。

9. mini_red_team.py 的 ASR 矩阵是什么意思。
10. mini_defense.py 为什么要比较 defense 前后。
11. portfolio.py 为什么是最终交付物，而不是附属文件。
12. 如果一个模型 Arena 排名第一但 red-team ASR 高，你会怎样写部署建议。
13. 如果 judge 偏好长回答，你会怎样改 rubric 或加 length control。
14. 如何让 AI agent 帮你审查 portfolio，而不是替你写空泛总结。

真正掌握的标志是: 你能从 5 个 checkpoint 出发，设计一份包含能力、偏好、安全、效率和部署建议的评测 portfolio，并能解释每一个分数背后的假设和风险。